Chapter 4: Database Design

4.1 Overview

PID-Guard uses a **single-tier relational database** architecture powered by **SQLite**, accessed asynchronously through **SQLAlchemy 2.0 (async)** with the aiosqlite driver. Unlike traditional deployments that require a separate database server (such as MySQL or PostgreSQL), SQLite stores its entire database in a single file on disk (pid_guard.db). This makes PID-Guard completely self-contained — no installation, configuration, or credentials are required to run the system, which is intentional for a student research environment.

The data layer is split into two logical levels that work together:

Layer	Technology	Role
Persistence	SQLite (pid_guard.db)	Stores all detection records on disk
ORM / Access	SQLAlchemy 2.0 Async + aiosqlite	Maps Python objects to SQL rows; provides async session management
Validation	Pydantic v2	Validates and serialises all API request/response payloads

4.2 Entity Descriptions

4.2.1 DETECTION_RECORD

The DETECTION_RECORD entity is the central and only table in the PID-Guard database. It is written to every time a user submits a prompt for analysis. Once written, records are **read-only**; there is no update or delete operation in the system. The table serves both as an audit trail and as the data source for the Dashboard analytics view.

Field Name	SQLAlchemy Type	SQL Type	Constraint	Description
id	Mapped[int]	INTEGER	PRIMARY KEY, AUTO-INCREMENT, INDEXED	Unique auto-incrementing identifier assigned at insert time.
prompt	Mapped[str]	VARCHAR(5000)	NOT NULL	The raw prompt text submitted by the user, up to 5,000 characters.

Field Name	SQLAlchemy Type	SQL Type	Constraint	Description
risk_score	Mapped[float]	REAL	NOT NULL	Final composite risk score in the range 0.0–100.0, computed by the detection engine.
status	Mapped[str]	VARCHAR(20)	NOT NULL	Classification result. One of: Safe, Suspicious, Injection.
drift_score	Mapped[float]	REAL	DEFAULT 0.0	One of: Financial Fraud, Harassment, Data Breach, Bribery, Safety Violation, Other.
behavior_score	Mapped[float]	REAL	DEFAULT 0.0	Behavioural pattern match score (0.0–100.0). Reflects the weight of matched injection-pattern keywords found in the prompt.
explanation	Mapped[str]	VARCHAR(2000)	DEFAULT ' '	Human-readable natural language explanation of why this risk score was assigned. Generated programmatically by the detection engine.
patterns_matched	Mapped[str]	VARCHAR(1000)	DEFAULT '[ ]'	JSON-serialised list of matched attack category names (e.g. ["instruction_override", "system_prompt_leak"]). Stored as a string due to SQLite's lack of a native array type.
timestamp	Mapped[datetime]	DATETIME	DEFAULT func.now()	UTC datetime recorded automatically at the moment the record is inserted.

4.3 API Data Models (Pydantic)

PID-Guard uses Pydantic v2 models to validate all HTTP request and response payloads. These models are defined in models.py and are logically separate from the SQLAlchemy ORM models, though they share the same field semantics.

4.3.1 DetectionRequest

Received by the POST /detect endpoint.

Field Name	Data Type	Constraint	Description
prompt	str	min_length=1, max_length=5000	The prompt text to analyse. Must be non-empty.

4.3.2 DetectionResponse

Returned by POST /detect after analysis and persistence.

Field Name	Data Type	Description
id	int (optional)	Database-assigned primary key of the saved record.
prompt	str	The original submitted prompt.
risk_score	float (0–100)	Final composite risk score.
status	str	One of: Safe, Suspicious, Injection.
drift_score	float (0–100)	Semantic intent drift component score.
behavior_score	float (0–100)	Behavioural pattern component score.
explanation	str	Human-readable detection rationale.
patterns_matched	List[str]	Attack category names matched in the prompt.
timestamp	datetime (optional)	UTC timestamp of the record.

4.3.3 HistoryItem

Returned by GET /history for each record in the audit log. Identical fields to DetectionResponse but with id and timestamp always present.

Field Name	Data Type	Constraint	Description
cid	string	PRIMARY KEY	The IPFS Content Identifier (CID) hash, e.g. QmXoyp…. This is a cryptographic fingerprint of the file.
fileSize	integer (bytes)	NULLABLE	Size of the uploaded evidence file in bytes, recorded at the time of upload.
uploadTimestamp	uint256 (integer)	NOT NULL	Unix epoch timestamp of when the file was pinned to IPFS.

4.3 Entity Relationships

The following table defines the relationships between entities using Crow's Foot Notation rules. These relationships are enforced programmatically by the smart contract logic rather than by a relational database engine.

Relationship	From Entity	To Entity	Cardinality & Description
submits	WHISTLEBLOWER	REPORT	One-to-Many (1 to 0..*). A whistleblower may submit zero or many reports. Each report belongs to exactly one whistleblower address.
updates_status_of	AUDITOR	REPORT	One-to-Many (1 to 0..*). An auditor may update the status of zero or many reports. Each status update is tied to one auditor's transaction.
references	REPORT	IPFS_STORAGE	One-to-Zero-or-One (1 to 0..1). A report may reference zero or one evidence file on IPFS. If no file is attached, ipfsHash is an empty string.

4.4 Entity-Relationship Diagram

Because PID-Guard contains a single table, the ER diagram shows the entity and all its attributes. The patterns_matched field represents an embedded one-to-many relationship (a single record can match multiple attack categories), serialised as a JSON string rather than a separate join table — a deliberate design trade-off for simplicity.

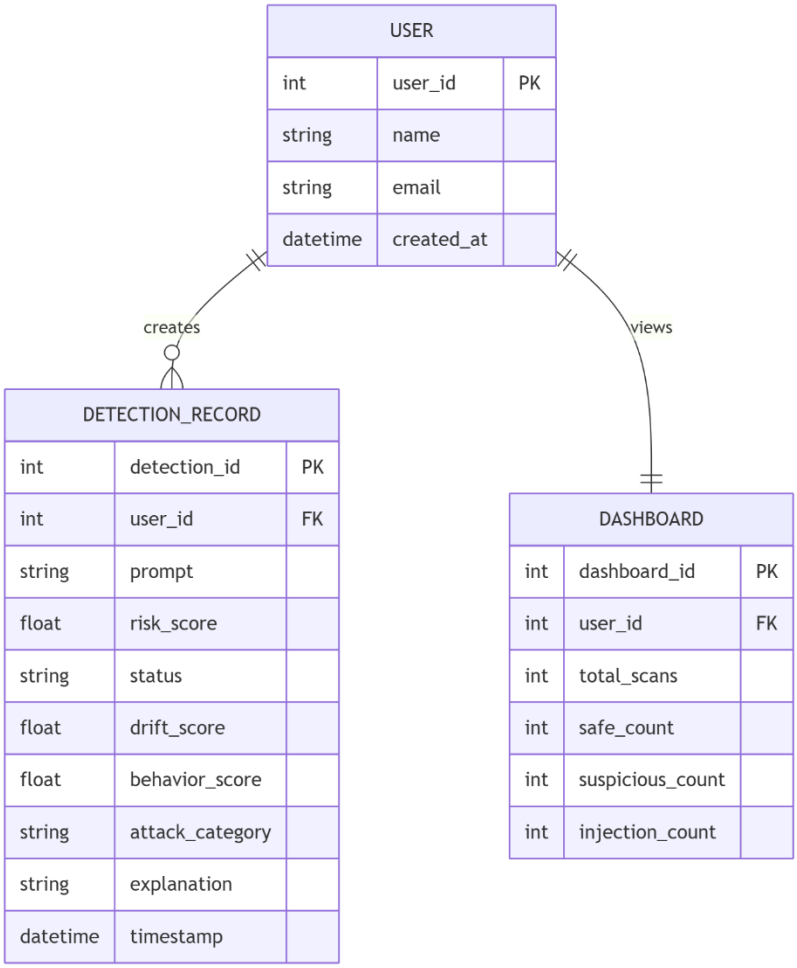

Figure 4.1: PID-Guard Entity-Relationship Diagram

To generate this diagram, paste the following Mermaid.js code into the Mermaid Live Editor at https://mermaid.live and export the result as a PNG image to include in this report.

```
erDiagram

    USER {

        int user_id PK
        string name
        string email
        datetime created_at

    }

    DETECTION_RECORD {

        int detection_id PK
        int user_id FK
        string prompt
```

```
        float risk_score
        string status
        float drift_score
        float behavior_score
        string attack_category
        string explanation
        datetime timestamp

    }

    DASHBOARD {

        int dashboard_id PK
        int user_id FK
        int total_scans
        int safe_count
        int suspicious_count
        int injection_count

    }

    USER ||--o{ DETECTION_RECORD : creates

    USER ||--|| DASHBOARD : views
```

4.5 CRUD Operations Reference

All database interaction is performed through crud.py, which contains two asynchronous functions. There are intentionally **no update or delete operations** — detection records are append-only to preserve the full audit history.

Operation	Function	SQL Equivalent	Triggered By
Create	save_detection(db, result, prompt)	INSERT INTO detections ...	POST /detect endpoint after every successful analysis
Read	get_history(db, limit=50)	SELECT * FROM detections ORDER BY timestamp DESC LIMIT n	GET /history endpoint; also used by Dashboard component

---

4.6 Design Justification

The database architecture of PID-Guard was designed based on key principles of simplicity, performance, security, and portability. The major design justifications are explained below.

Zero-Configuration Setup

PID-Guard uses SQLite as its database engine because it requires no separate server process, credentials, or installation beyond the Python *aiosqlite* package. The database file, **pid_guard.db**, is automatically created during the first execution using the Base.metadata.create_all() function. This approach ensures that the system can be deployed and executed easily on any standard laptop without requiring DevOps expertise or complex configuration. This is particularly suitable for student research projects and prototype systems.

Asynchronous I/O Compatibility

The backend of PID-Guard is built using FastAPI, which operates on the ASGI standard and supports asynchronous request handling using Python's asyncio framework. Using a synchronous SQLAlchemy engine would block the event loop during database operations, thereby reducing system performance and scalability. To address this, PID-Guard uses create_async_engine and AsyncSession, which enable non-blocking database operations. This ensures efficient concurrent processing and seamless integration with FastAPI's asynchronous architecture.

Append-Only Audit Trail

The Detection_Record table follows an append-only design, where records are never modified or deleted after insertion. Each prompt analysis generates a permanent, timestamped record. This design follows the immutability principle commonly used in audit logging systems. It ensures data integrity and provides a reliable historical log of all prompt detection activities, which is essential for security analysis, research validation, and forensic review.

JSON Serialization of patterns_matched

SQLite does not provide native support for array or JSON column types. To overcome this limitation, the **patterns_matched** field stores data as a JSON-encoded string. The list of detected patterns is serialized using Python's json.dumps() method during insertion and deserialized using json.loads() during retrieval. This approach preserves detailed categorical information without requiring additional relational tables, thereby simplifying the database schema while maintaining flexibility.

Separation of Concerns Using Pydantic and SQLAlchemy

PID-Guard maintains a clear separation between the database layer and the API layer. The SQLAlchemy ORM model (**DetectionRecord**) is used exclusively for database operations, while the Pydantic model (**DetectionResponse**) is used for API responses. This design follows the Repository Pattern, ensuring modularity and maintainability. Any future database migration, such as moving from SQLite to PostgreSQL, can be performed by modifying only the database layer without affecting the API interface.

Portability and Replaceability

The database connection string is configured using the **DATABASE_URL** environment variable, with a default value of sqlite+aiosqlite:///./pid_guard.db. This design allows easy migration to production-grade databases such as PostgreSQL by simply changing the environment variable, without requiring any changes to the application code. SQLAlchemy provides abstraction over the database engine, ensuring portability and long-term scalability.